EXPERIMENT – 3

TITLE

Build a Regression Model Using TensorFlow for Predicting House Prices

Name: Sujay V
USN: 24BTRAO041
Course: MLOPS

1. AIM

To build and train a regression model using TensorFlow and Keras for predicting house prices based on different house-related features such as area, number of bedrooms, number of bathrooms, number of stories, and age of the house.

The experiment demonstrates the complete machine learning workflow including dataset loading, data analysis, preprocessing, feature scaling, train-test splitting, neural network model development, model training, evaluation, prediction, and visualization of actual and predicted house prices.

2. DATASET DESCRIPTION

The dataset used for this experiment is **house_prices.csv**. The dataset contains **300 records and 6 columns**. All six columns contain integer values and there are no missing values in the dataset.

The dataset contains the following attributes:

area: Represents the area of the house in square feet.

bedrooms: Represents the number of bedrooms in the house.

bathrooms: Represents the number of bathrooms in the house.

stories: Represents the number of stories in the house.

age: Represents the age of the house in years.

price: Represents the price of the house and is used as the target variable.

The input features used for prediction are **area, bedrooms, bathrooms, stories, and age**, while **price** is considered the target variable.

Since house price is a continuous numerical value, predicting the price is considered a **regression problem**.

The dataset was checked for missing values and all six columns were found to contain **zero missing values**.

The dataset was divided into **80% training data and 20% testing data**. This resulted in **240 training samples and 60 testing samples**.

The input features were standardized using **StandardScaler**. The target variable was also scaled before training the neural network to improve the training process.

3. PYTHON CODE AND OUTPUT SCREENSHOTS

The house price prediction model was implemented using **Python in Jupyter Notebook**.

The required libraries including **Pandas, NumPy, Matplotlib, Scikit-learn, and TensorFlow Keras** were imported for data processing, visualization, preprocessing, and neural network development.

The dataset was loaded using Pandas and the first five records were displayed. The dataset shape was found to be **(300, 6)**.

The dataset structure was then analyzed using data.info(). All six columns contained **300 non-null values** and all columns were of integer data type.

The missing-value check showed that there were **no missing values** in any of the columns.

The input features and target variable were separated. The five input features selected were **area, bedrooms, bathrooms, stories, and age**, while **price** was selected as the target variable.

The dataset was split into training and testing sets using an **80:20 ratio**. The training set contained **240 samples**, while the testing set contained **60 samples**.

The input features were standardized using StandardScaler. The scaler was fitted using the training data and then applied to the testing data.

The target variable was also standardized using another StandardScaler. This scaled target data was used during the training process.

A **TensorFlow Keras Sequential neural network** was then created. The model consists of an input layer, a Dense layer containing **64 neurons**, a second Dense layer containing **32 neurons**, and an output layer containing **1 neuron** for predicting the house price.

The model summary showed a total of **2,497 trainable parameters**, with no non-trainable parameters.

The model was compiled using the **Adam optimizer** with a learning rate of **0.001**. The **Mean Squared Error (MSE)** was used as the loss function and **Mean Absolute Error (MAE)** was used as the evaluation metric.

The model was trained for **200 epochs** using a batch size of **16**. A validation split of **20%** was used during training to monitor the model's validation performance.

The training and validation loss were plotted against the number of epochs. This graph helps to visualize the learning behavior of the model during training.

The trained model was evaluated on the test dataset. The scaled test loss was **0.1092**, while the scaled Mean Absolute Error was **0.2648**.

Predictions were generated for the test dataset and then converted back to the original house-price scale using the inverse transformation of the target scaler.

Finally, the actual and predicted prices were compared using a table and a scatter plot. The scatter plot contains a reference line representing perfect prediction.

P . T . O

In [16]:
```
# Sujay V
# 24BTRAO041

# Step 1: Import required libraries

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
```

In [17]:
```
# Step 2: Load the house price dataset

data = pd.read_csv(r"C:\Users\Sujay\OneDrive\Documents\GitHub\House_Price_Predic

print("Dataset loaded successfully!")
print("Dataset Shape:", data.shape)

data.head()
```

```
Dataset loaded successfully!
Dataset Shape: (300, 6)
```

Out[17]:

	area	bedrooms	bathrooms	stories	age	price
0	3674	4	3	2	16	842100
1	1360	1	1	2	27	300000
2	1794	1	1	2	25	329000
3	1630	2	3	1	8	478500
4	1595	1	1	2	21	300000

In [18]:
```
# Step 3: Display the structure, data types and number of non-null values

data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 300 entries, 0 to 299
Data columns (total 6 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   area       300 non-null    int64
 1   bedrooms   300 non-null    int64
 2   bathrooms  300 non-null    int64
 3   stories    300 non-null    int64
 4   age        300 non-null    int64
 5   price      300 non-null    int64
dtypes: int64(6)
memory usage: 14.2 KB
```

In [19]:
```
# Step 4: Check whether the dataset contains any missing or null values

print("Missing Values in Each Column:")
print(data.isnull().sum())
```

```
Missing Values in Each Column:
area         0
bedrooms     0
bathrooms    0
stories      0
age          0
price        0
dtype: int64
```

In [20]:
```
# Step 5: Separate the input features from the target variable

X = data.drop("price", axis=1)
y = data["price"]

print("Input Features:")
print(X.columns.tolist())

print("\nTarget Variable:")
print("price")
```

```
Input Features:
['area', 'bedrooms', 'bathrooms', 'stories', 'age']

Target Variable:
price
```

In [21]:
```
# Step 6: Split the dataset into 80% training data and 20% testing data

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("Training Data:", X_train.shape)
print("Testing Data:", X_test.shape)
```

```
Training Data: (240, 5)
Testing Data: (60, 5)
```

In [22]:
```
# Step 7: Scale the input features using StandardScaler

X_scaler = StandardScaler()

X_train_scaled = X_scaler.fit_transform(X_train)
X_test_scaled = X_scaler.transform(X_test)

print("Input features scaled successfully!")
```

```
Input features scaled successfully!
```

In [23]:
```
# Step 8: Scale the target variable for better neural network training

y_scaler = StandardScaler()
```

```
y_train_scaled = y_scaler.fit_transform(
    y_train.to_numpy().reshape(-1, 1)
).flatten()

y_test_scaled = y_scaler.transform(
    y_test.to_numpy().reshape(-1, 1)
).flatten()

print("Target variable scaled successfully!")
```

```
Target variable scaled successfully!
```

In [24]:
```
# Step 9: Build the regression model using TensorFlow Keras

model = keras.Sequential([
    keras.Input(shape=(X_train_scaled.shape[1],)),
    layers.Dense(64, activation="relu"),
    layers.Dense(32, activation="relu"),
    layers.Dense(1)
])

model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_3 (Dense)	(None, 64)	384
dense_4 (Dense)	(None, 32)	2,080
dense_5 (Dense)	(None, 1)	33

Total params: 2,497 (9.75 KB)

Trainable params: 2,497 (9.75 KB)

Non-trainable params: 0 (0.00 B)

In [25]:
```
# Step 10: Compile the model using Adam optimizer, MSE loss and MAE metric

model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss="mse",
    metrics=["mae"]
)

print("Model compiled successfully!")
```

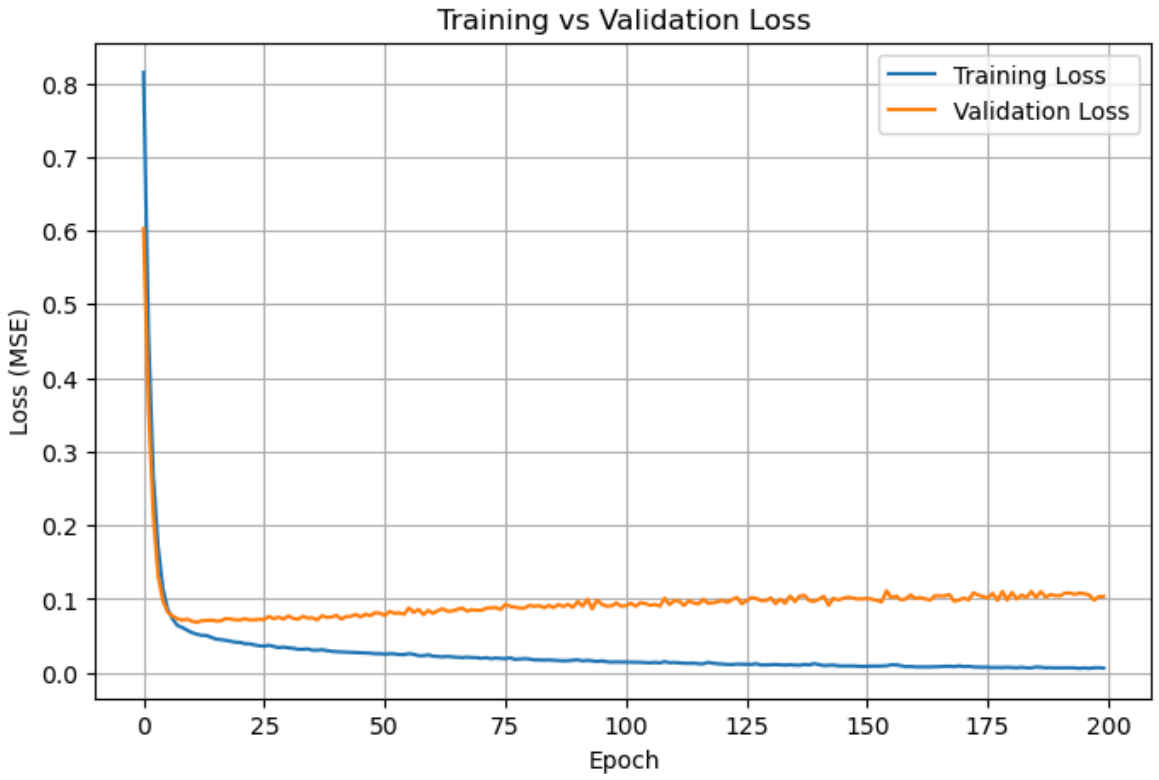
```
Model compiled successfully!
```

In [26]:
```
# Step 11: Train the model using the training dataset

history = model.fit(
    X_train_scaled,
    y_train_scaled,
    validation_split=0.2,
    epochs=200,
    batch_size=16,
    verbose=0
```

```
)

print("Model training completed successfully!")
```

```
Model training completed successfully!
```

In [27]:
```
# Step 12: Plot training loss and validation loss

plt.figure(figsize=(8, 5))

plt.plot(
    history.history["loss"],
    label="Training Loss"
)

plt.plot(
    history.history["val_loss"],
    label="Validation Loss"
)

plt.xlabel("Epoch")
plt.ylabel("Loss (MSE)")
plt.title("Training vs Validation Loss")
plt.legend()
plt.grid(True)

plt.show()
```



In [28]:
```
# Step 13: Evaluate the trained model on the test data

test_loss, test_mae = model.evaluate(
    X_test_scaled,
    y_test_scaled,
    verbose=0
)
```

```
print("Test Loss (MSE):", test_loss)
print("Test MAE:", test_mae)
```

```
Test Loss (MSE): 0.10921382158994675
Test MAE: 0.26478809118270874
```

In [29]:
```
# Step 14: Generate predictions for the test dataset

predictions_scaled = model.predict(
    X_test_scaled,
    verbose=0
)

# Convert predictions back to original price scale
predictions = y_scaler.inverse_transform(
    predictions_scaled
).flatten()

print("Predictions generated successfully!")
```

```
Predictions generated successfully!
```

In [30]:
```
# Step 15: Calculate MSE, MAE, RMSE and R2 score

mse = mean_squared_error(
    y_test,
    predictions
)

mae = mean_absolute_error(
    y_test,
    predictions
)

rmse = np.sqrt(mse)

r2 = r2_score(
    y_test,
    predictions
)

print("Model Evaluation Results:")
print("Mean Squared Error (MSE):", round(mse, 2))
print("Mean Absolute Error (MAE):", round(mae, 2))
print("Root Mean Squared Error (RMSE):", round(rmse, 2))
print("R2 Score:", round(r2, 4))
```

```
Model Evaluation Results:
Mean Squared Error (MSE): 3167534592.0
Mean Absolute Error (MAE): 45094.16
Root Mean Squared Error (RMSE): 56280.85
R2 Score: 0.8842
```

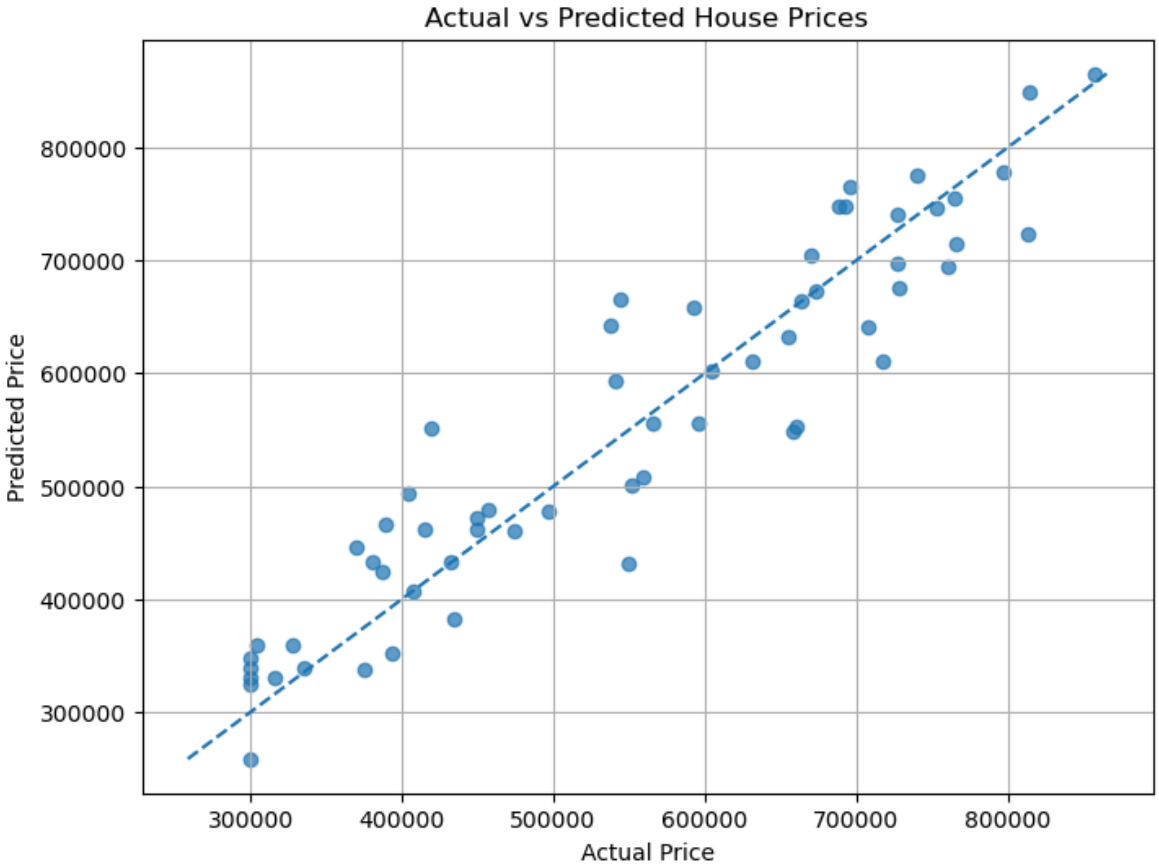
In [31]:
```
# Step 16: Compare actual and predicted house prices

comparison = pd.DataFrame({
    "Actual Price": y_test.to_numpy(),
    "Predicted Price": predictions
})

comparison.head(10)
```

Out[31]:

	Actual Price	Predicted Price
0	369300	445362.81250
1	386500	424716.15625
2	497000	476994.21875
3	380500	433380.25000
4	813900	849101.62500
5	474100	460291.46875
6	766000	715166.68750
7	537600	641627.00000
8	765000	755371.81250
9	603800	601570.93750

In [32]:

```
# Step 17: Visualize actual prices against predicted prices

plt.figure(figsize=(8, 6))

plt.scatter(
    y_test,
    predictions,
    alpha=0.7
)

min_price = min(y_test.min(), predictions.min())
max_price = max(y_test.max(), predictions.max())

plt.plot(
    [min_price, max_price],
    [min_price, max_price],
    linestyle="--"
)

plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.title("Actual vs Predicted House Prices")

plt.grid(True)

plt.show()
```



In [33]:
```python
# Step 18: Display the first 10 actual and predicted prices

print("First 10 Predictions:")

for actual, predicted in zip(
    y_test.to_numpy()[:10],
    predictions[:10]
):
    print(
        f"Actual: {actual:.2f} | Predicted: {predicted:.2f}"
    )
```

```
First 10 Predictions:
Actual: 369300.00 | Predicted: 445362.81
Actual: 386500.00 | Predicted: 424716.16
Actual: 497000.00 | Predicted: 476994.22
Actual: 380500.00 | Predicted: 433380.25
Actual: 813900.00 | Predicted: 849101.62
Actual: 474100.00 | Predicted: 460291.47
Actual: 766000.00 | Predicted: 715166.69
Actual: 537600.00 | Predicted: 641627.00
Actual: 765000.00 | Predicted: 755371.81
Actual: 603800.00 | Predicted: 601570.94
```

4. MODEL EVALUATION RESULTS

The trained TensorFlow regression model was evaluated using **Mean Squared Error (MSE), Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R^2 Score**.

The dataset consisted of 300 samples, out of which **240 samples were used for training and 60 samples were used for testing**.

REGRESSION PERFORMANCE

Metric	**Result**
Mean Squared Error (MSE)	**3,167,534,592.00**
Mean Absolute Error (MAE)	**45,094.16**
Root Mean Squared Error (RMSE)	**56,280.85**
R^2 Score	**0.8842**

The model achieved an **R^2 Score of 0.8842**, indicating that the model explains approximately **88.42% of the variation** in the house prices in the test dataset.

The **Mean Absolute Error of 45,094.16** indicates that the predicted house prices differ from the actual prices by approximately 45,094 price units on average.

The **Root Mean Squared Error of 56,280.85** indicates the typical magnitude of prediction error while giving greater importance to larger errors.

The Mean Squared Error obtained on the original price scale was **3,167,534,592.00**.

ACTUAL VS PREDICTED PRICES

The model generated predictions for the 60 test samples and compared them with their actual house prices.

The first ten predictions demonstrate that the model produces reasonably close predictions for several houses. For example, the actual price of **369,300** was predicted as **445,362.81**, while an actual price of **813,900** was predicted as **849,101.62**.

Some of the first ten prediction results are shown below:

Actual Price	**Predicted Price**
369300	445362.81
386500	424716.16
497000	476994.22

380500	433380.25
813900	849101.62
474100	460291.47
766000	715166.69
537600	641627.00
765000	755371.81
603800	601570.94

The actual and predicted values were also visualized using a scatter plot. Points closer to the reference line represent predictions that are closer to the actual house prices.

TRAINING AND VALIDATION LOSS

The training and validation loss graph was generated to observe the model's performance throughout the 200 training epochs.

The graph provides a visual representation of the change in Mean Squared Error during training and validation. It helps in understanding how the neural network learns from the training data and performs on validation data.

5. GITHUB REPOSITORY LINK

The complete implementation of the House Price Prediction model is available on GitHub.

GitHub Repository:
https://github.com/sujayv26/Tensorflow-house-price-prediction.git

6. CONCLUSION

The TensorFlow-based regression model was successfully implemented to predict house prices using features such as area, bedrooms, bathrooms, stories, and age.

The experiment demonstrated the complete machine learning workflow, including dataset loading, preprocessing, feature scaling, train-test splitting, neural network model creation, model training, evaluation, prediction, and visualization.

The model was built using TensorFlow and Keras with two hidden layers containing 64 and 32 neurons and one output neuron. The Adam optimizer, Mean Squared Error loss function, and Mean Absolute Error metric were used for training and evaluation.

The model performance was evaluated using MSE, MAE, RMSE, and R^2 Score. The actual and predicted house prices were also visualized to understand the prediction performance.

Thus, the experiment demonstrates the practical application of **TensorFlow and neural network regression for predicting continuous numerical values such as house prices**.